

Agent Hub — Plan Review Session Record

Claude Opus 5 · /plan-ceo-review + /plan-eng-review
2026-07-28

CONTENTS

Agent Hub — Plan Review Session Record

What ran

The findings that changed the plan

1. The proposed reply-detection fix did not fix the bug it was written for

2. Edit pairs were being destroyed, not discarded — which inverted the build order

3. Making the watcher long-lived would have activated a dormant refresh storm

4. The dedupe key was the message body text

5. Two defects existed only in the combination of approved decisions

6. Five load-bearing modules were not in git

Decisions of record

What was written

The one code change

What is still open

Recommended next steps

Agent Hub — Plan Review Session Record

Date: 2026-07-27 to 2026-07-28 **Branch:** feat/outreach-drafting-optout **Subject:** the daily-use build — reply detection, background watcher, notification, fast approval queue **Outcome:** 41 decisions taken, 0 unresolved, 72 implementation tasks emitted, 1 live repo defect found and fixed

What ran

Pass	Skill	Result
1	/plan-ceo-review	HOLD SCOPE, all 7 phases held, 23 decisions, 39 tasks
2	Outside voice (Claude subagent)	14 findings, 4 correct catches the review missed
3	/plan-eng-review	18 findings, 15 decisions, 33 tasks
4	Outside voice (Claude subagent)	15 findings, including the 2 best of the cycle

Codex was not installed, so both outside-voice passes ran as independent Claude subagents with fresh context. Tests were 228/228 passing before and after every pass. Neither review changed application code — that was the point of running them.

The findings that changed the plan

1. THE PROPOSED REPLY-DETECTION FIX DID NOT FIX THE BUG IT WAS WRITTEN FOR

The plan called for replacing newest-only detection with “scan all messages after our last outgoing message.” Traced against the actual failure case, that does nothing: when the message that landed since is the *operator’s own manual Gmail reply*, the contact’s reply is still behind the boundary. The fix would have shipped, met its written definition of done, and kept losing warm replies.

Any boundary derived from message ordering has this hole. The rule is now dedupe-driven — the newest message from the contact that has no review row yet — because the review table is authoritative and message order is not.

2. EDIT PAIRS WERE BEING DESTROYED, NOT DISCARDED — WHICH INVERTED THE BUILD ORDER

The plan described capturing draft edits as future work. In fact `drafts_db.mark_sent` and `reviews_db.mark_sent` **overwrite** the model’s original in place at send time. The training corpus was being deleted daily and cannot be backfilled.

That made capture (two columns, two write sites) the only item in a seven-phase plan that gets more expensive every day it waits. It moved from last to first.

3. MAKING THE WATCHER LONG-LIVED WOULD HAVE ACTIVATED A DORMANT REFRESH STORM

`google_auth.get_credentials` persists a refreshed token to the database but never updates the account dict it was handed, and `accounts_db.get_google_token` reads the token *from that dict*. A loop holding a bound dict therefore re-reads a stale token every cycle, forces an OAuth refresh, writes the DB, and never sees its own write — 288 forced refreshes and 288 wasted writes per account per day at a 5-minute interval.

The bug does not exist today because the script is short-lived. Phase 1 is precisely the change that activates it.

4. THE DEDUPE KEY WAS THE MESSAGE BODY TEXT

`find_review_id` matched on exact `customer_reply`. Once detection walked every contact message instead of just the newest, that key failed three ways at once: two “sounds good” messages on one thread collide and the second never surfaces; a multi-KB body in a PostgREST `.eq()` is a GET parameter that eventually 414s; and Postgres btree caps index entries near 2704 bytes, so the race could never be closed with a unique constraint.

Keying on the Gmail message id — stable, short, already in the payload — collapsed the collision, the URL-length failure, and the detector race into one column.

5. TWO DEFECTS EXISTED ONLY IN THE *COMBINATION* OF APPROVED DECISIONS

Neither review would have caught these by checking decisions one at a time.

Double-reply. The dedupe rule fixed missed replies and created a new failure: the operator answers a prospect from the Gmail app at 11pm, the message still has no review row, the next cycle drafts and notifies, and the operator sends a second reply at 3.6 seconds a draft. Fixed with an `answered_elsewhere` state — visible, no draft, no notification.

Stranded reviews. Using the workflow status as the queue’s visibility filter was fine. Adding a validator with a regeneration loop was fine. Together they made a user-facing notification conditional on the LLM being up: a rejected draft strands the review and the user is never told a prospect replied. Visibility is now separate from workflow state.

6. FIVE LOAD-BEARING MODULES WERE NOT IN GIT

Found while assessing “land the branch.” `billing.py`, `bounces.py`, `dns_check.py`, `plans.py` and `providers.py` are imported by `server.py`, `agent.py`, `send_outreach.py`, `watch_replies.py`, `sheets.py`, `leads.py` and the test suite. Git had no copy of any of them, and nothing in `.gitignore` explained it — they were simply never added, across 51 commits.

Merging the branch into `main` would have produced a `main` that raises `ImportError` on boot and cannot run its own test suite. The 228 passing tests were passing against five files that existed only in one working directory.

Both reviews had sized the branch with `git diff main --stat`, which counts tracked files only. The modules were structurally invisible to the measurement.

Decisions of record

Forty-one decisions were taken. The load-bearing ones:

Detection

- Rule is dedupe-driven, keyed on the Gmail message id, not the body text
- Sender classification runs *before* reply candidacy, precedence `daemon > auto-responder > on-sheet > off-sheet` — otherwise `find_bounce` (gated on `if not reply_text`) becomes unreachable and bounce suppression dies silently
- Off-sheet senders get a flagged review with no draft until the user confirms
- A contact message our own message postdates is `answered_elsewhere`
- Older unreviewed contact messages are marked superseded

Worker

- Standalone process, Docker Compose for server plus worker
- One `scheduled_jobs` table replaces the `schedule` dependency, a global heartbeat row, and a job registry — per-account heartbeat, so one account wedged behind a dead token is visible rather than hidden behind a healthy tick
- Detection split from drafting; drafting claims rows with a conditional update and a reclaim sweep for stranded claims
- The worker iterates account IDs and re-reads each row fresh per cycle

Safety

- `SESSION_SIGNING_KEY` is minted new; the Fernet derivation stays on the existing `APP_SECRET_KEY`, so no stored token is re-keyed and no account is forced to reconnect
- `_validate_reply` plus a regeneration loop plus eval cases — the untrusted path had strictly weaker defences than the trusted one
- Phase 3's fast queue is gated on that validator being green, because 3.6 seconds a draft converts human approval from a real control into a nominal one

- Web Push kept, paired with the existing `notify.py` for the closed-laptop case push structurally cannot serve
- Push endpoint host allowlist at subscribe time (the client supplies the URL the server POSTs to)

Product

- The “N sends left today” digest line was dropped as manufactured urgency, against the plan’s own stated anti-goals
- Phase 6b (auto-suggested instruction changes) deferred; its gate cannot be met until an account has 30+ captured edit pairs

What was written

Artifact	Contents
<code>TODOS.md</code>	9 new entries, 3 existing entries corrected
<code>Agent-Hub-Daily-Use-Design.md</code>	Build-order corrections from both reviews, plus the review report
<code>tasks-ceo-review-20260727-222636.jsonl</code>	39 tasks, 26 P1
<code>tasks-eng-review-20260728-170620.jsonl</code>	33 tasks, 22 P1
Eng review test plan	QA-consumable, affected routes, edge cases, critical paths
Learnings	6 durable entries, 9/10 confidence

Three existing `TODOS.md` entries were corrected because they recommended approaches since proven wrong: the superseded scan-since-last-outgoing detector, the key-split method that would have forced a re-consent, and a reference to a `outreach_replies` table that does not exist (the table is `reviews`).

The one code change

`a5c7e80` — six files, +1,178 lines, nothing else. The five untracked modules plus `outreach-agent/tests/test_usage.py`. Committed alone so the history stays reviewable.

No application logic was modified during either review.

What is still open

Uncommitted: 31 modified tracked files, roughly +4,280/-254, including +2,227 lines of `tests/test_outreach_agent.py`. Never reviewed. This is the largest body of unreviewed code in the picture and it is what lands on `main`.

Still untracked: `outreach-agent/eval_models.py` (documented in the health stack, and now the home for the reply-validator eval), five `Agent-Hub-*.md` design docs including the one carrying both review reports, plus `.agents/`, `web/`, `e2e-shots/` and two log files.

Accepted and untracked by decision:

- Unattended detection is capped at 7 days per account until OAuth verification and the CASA assessment land. The validation period will run with a weekly reconnect for every candidate.
- Reply drafting has no volume limit and no unit meter. `monthly_replies` is `None` on all three plans and `usage.record_unit(UNIT_DRAFT_REPLY, ...)` is never called, so the counter reads zero permanently. The day anyone sets that limit to a number, it silently never fires.

Review status: the eng review logged `issues_open`, so `/ship` will not treat this branch as cleared until the P1 tasks land. That is correct.

Recommended next steps

1. Commit `eval_models.py` and the design docs. Two minutes; removes the last of the single-copy risk.

2. Run `/review` on the 31 modified files before committing them. Not for rigor's sake — because this session established that the branch's actual state and its assumed state had diverged badly, and that diff is the remaining place where that could still be true.
3. Merge (a clean fast-forward, 52 commits, zero conflicts), then start at E1 — dedupe on the Gmail message id — since the rest of the detector work sits behind it.

A third plan review would be planning a plan. Two passes found real defects; the next thing that finds a real defect is code.